

Informe de Pruebas - Práctica 5A

1. Introducción y Objetivos

El objetivo de este proceso de pruebas es verificar el correcto funcionamiento de la aplicación de gestión de seguros, asegurando tanto la integridad de la lógica de negocio en aislamiento (pruebas unitarias) como la correcta interacción entre las distintas capas del sistema, incluyendo la interfaz gráfica y la persistencia (pruebas de integración).

2. Fase 1: Pruebas Unitarias

En esta fase se han probado de forma aislada las clases del dominio: **Seguro** y **Cliente**.

2.1. Técnicas Utilizadas

- **Caja Negra:** Se han aplicado las técnicas de Partición Equivalente y Análisis de Valores Límite (AVL) para cubrir los distintos rangos de datos de entrada.
- **Caja Blanca:** Se ha buscado alcanzar la cobertura de sentencias y cobertura de decisión/condición mediante el uso de la herramienta Eclemma.

2.2. Diseño de Casos de Prueba para la Clase Seguro

Se ha testado el método `precio()` considerando las variables de cobertura, potencia, fecha de inicio y antigüedad del seguro [Seguro.java].

Identificador	Descripción	Técnica	Resultado Esperado
precioEsCeroSiFechaInicioEsNula	Fecha de inicio no definida	Caja Negra	0.0

Identificador	Descripción	Técnica	Resultado Esperado
precioEsCeroSiCoberturaEsNula	Cobertura no definida	Caja Negra	0.0
precioEsCeroSiFechaInicioEsPosteriorAHoy	Seguro no entrado en vigor	AVL	0.0
precioTercerosSinIncrementoConPotenciaMenorA90	Potencia en límite inferior (89)	AVL	400.0
precioTercerosConIncrementoCincoPorcentajeEnLimiteInferior	Potencia en frontera (90)	AVL	420.0
precioTercerosConIncrementoCincoPorcentajeEnLimiteSuperior	Potencia en frontera (110)	AVL	420.0
precioTercerosConIncrementoVeintePorcentajeCuandoPotenciaMayorA110	Potencia fuera de rango (111)	AVL	480.0

Identificador	Descripción	Técnica	Resultado Esperado
precioTodoRiesgoConDescuentoDurantePrimerAño	Aplicación de descuento del 20%	Partición	800.0
precioTercerosLunasSinDescuentoEnAniversario	Sin descuento tras el primer año	AVL	600.0

2.3. Diseño de Casos de Prueba para la Clase Cliente

Se ha testado el método totalSeguros() para verificar el sumatorio de precios y la aplicación del descuento por minusvalía [Cliente.java].

Identificador	Descripción	Resultado Esperado
totalSegurosEsCeroCuandoNoTieneSeguros	Lista de seguros vacía	0.0
totalSegurosSumaImportesSinDescuentoCuandoNoHayMinusvalia	Suma estándar de seguros	150.5
totalSegurosAplicaDescuentoPorMinusvaliaASeguro	Aplicación del 25% de descuento	225.0

2.4. Análisis de Cobertura de Caja Blanca

Tras la ejecución de las pruebas anteriores, se ha verificado con EclEmma que se alcanza el 100% de cobertura de sentencias y de ramas en ambas clases, cumpliendo así con los criterios de aceptación.

3. Fase 2: Pruebas de Integración

El objetivo es comprobar el funcionamiento conjunto de la clase **VistaAgente** con la capa de negocio (GestionSeguros) y la capa de acceso a datos (DAO) utilizando una base de datos H2 en memoria.

3.1. Entorno de Pruebas

- **Herramientas:** JUnit y AssertJ Swing para la interacción con la interfaz gráfica.
- **Estado inicial de la BD:** Se han utilizado los datos precargados definidos en el anexo de la práctica (Juan, Ana, Luis, Pepe).
- **Configuración:** Las clases de prueba utilizan el sufijo IT y se ejecutan en la fase verify de Maven.

3.2. Diseño de Casos de Prueba para VistaAgente

Se ha automatizado la interacción con la ventana para el caso de uso "Consulta Cliente" [VistaAgentelT.java].

Identificador	Escenario	Acciones y Verificaciones
consultaClienteValidoMuestraNombreTotalYSeguros	Cliente con seguros (Juan)	DNI "11111111A": Nombre "Juan", Total "1820.0", 3 seguros.
consultaClienteInexistenteMuestraMensajeYLimpiaDatos	DNI no registrado	DNI "00000000Z": "DNI No Válido", campos limpios.
consultaClienteSinSeguros	Cliente sin seguros (Luis)	DNI "33333333A": Nombre

Identificador	Escenario	Acciones y Verificaciones
MuestraTotalCeroYListaVacía		"Luis", Total "0.0", lista vacía.

4. Conclusiones

El proceso de pruebas ha permitido validar la robustez de la lógica de cálculo de precios y la correcta integración de los componentes del sistema. El uso de herramientas de automatización como AssertJ Swing asegura que la interfaz gráfica responde correctamente a los datos proporcionados por las capas inferiores. Todas las pruebas diseñadas se han ejecutado satisfactoriamente.